

Vamos passar pelas nossas quatro grandes etapas: desde os fundamentos de interfaces híbridas e multiplataforma até a implementação de um sistema robusto com banco de dados na nuvem e comunicação em tempo real (*Realtime*).

Bons estudos, e aproveitem o material!

Módulo 5 (Material 05): Fundamentos e Interface Mobile (Nativo vs. Híbrido vs. Cross-Platform)

Para começarmos, precisamos entender as engenharias por trás dos aplicativos que usamos diariamente. O mercado se divide essencialmente em três abordagens de desenvolvimento:

1. Abordagens de Desenvolvimento

- **Nativo:** Desenvolvemos um código específico para cada plataforma. Usamos Swift/Objective-C para iOS e Kotlin/Java para Android. A grande vantagem é o **alto desempenho** e o acesso total aos recursos do sistema, mas o custo e o tempo de desenvolvimento são bem maiores.
- **Híbrido (Web View):** O aplicativo roda tecnologias web clássicas (HTML, CSS e JavaScript) dentro de um container nativo isolado (uma *WebView*). É muito rápido de desenvolver, mas o **desempenho é limitado** e há uma forte dependência de plugins para acessar recursos do aparelho. Exemplos: Cordova e as versões antigas do Ionic.
- **Cross-Platform (Compilado/Ponte):** É a abordagem que escolhemos para nossas aulas! Escrevemos o código em uma única linguagem (como JavaScript no React Native ou Dart no Flutter). Esse código é interpretado ou compilado para gerar interfaces e lógicas nativas reais. Ele busca o "melhor dos dois mundos": desenvolvimento rápido com uma única base de código e desempenho muito superior ao híbrido.

2. O Ecossistema React Native e Expo

Nas nossas aulas, o nosso ecossistema padrão é composto por:

- **React Native:** O framework criado pelo Facebook que nos permite construir apps usando JavaScript/JSX, renderizando componentes de interface estruturalmente nativos (não WebViews). Ele usa uma **ponte (Bridge)** para comunicar o código JavaScript com as APIs nativas do aparelho.
- **Expo:** Um conjunto de ferramentas fantástico construído em cima do React Native para simplificar nossa vida, acelerando o desenvolvimento e eliminando a necessidade inicial de configurar ambientes pesados como Android Studio ou Xcode.
- **Managed Workflow (Fluxo Gerenciado):** É o fluxo que adotamos. Nós nos preocupamos exclusivamente em escrever código JavaScript/TypeScript, deixando toda a complexidade da infraestrutura nativa por conta do Expo.

3. Diferenças Cruciais: Web vs. Mobile

Lembrem-se: **no React Native não usamos HTML**. Substituímos as tags tradicionais da web por componentes nativos. Além disso, **todo texto deve estar obrigatoriamente dentro da tag `<Text>`**, caso contrário o app gerará um erro imediato.

HTML (Web)	React Native	Descrição
<code><div></code>	<code><View></code>	O container ou bloco padrão de layout. Não possui rolagem automática.
<code><p>, , <h1></code>	<code><Text></code>	Utilizado para exibir qualquer texto na tela.
<code><button>, <a></code>	<code><TouchableOpacity></code>	Componente que detecta o toque do usuário e reduz a opacidade para dar feedback visual.
<code></code>	<code><Image></code>	Exibição de imagens (requer obrigatoriamente a propriedade <code>source</code>).
<code>onClick</code>	<code>onPress</code>	O evento disparado quando o usuário toca na tela.

Atenção com o Flexbox Mobile: Na Web, o padrão da propriedade `flexDirection` é `row` (linha). **No React Native, o padrão é `column` (coluna).** Desenhemos as interfaces verticalmente porque essa é a orientação natural de rolagem dos smartphones.

Módulo 6 [Material 06]: Otimização de Listas e Inputs (O Exemplo `Tasks.js`)

Quando lidamos com formulários e listagens no ambiente mobile, a performance é nossa prioridade regulamentar. Diferente da Web, os campos de texto no mobile não guardam o estado sozinhos de forma automatizada; precisamos gerenciar isso explicitamente via *hooks* (`useState`).

⚠️ Regra de Ouro para a Prova: Nunca utilizem o método `.map()` do JavaScript para renderizar listas longas ou dinâmicas no ambiente mobile. O `.map()` renderiza todos os elementos de uma vez na memória (mesmo os que estão fora da tela), o que pode travar o celular do usuário. Em vez disso, usem sempre o `<FlatList>`, que implementa *lazy loading* e renderiza apenas o que está visível na janela do aparelho.

Abaixo está o exemplo completo que construímos em aula (`src/screens/Tasks.js`) para fixarem o uso de estados estruturados, inputs e listas performáticas.

```
// src/screens/Tasks.js
import React, { useState } from 'react';
import { StyleSheet, Text, View, TextInput, Button, FlatList } from 'react-native';

export default function Tasks() {
  const [tarefa, setTarefa] = useState('');
  const [lista, setLista] = useState([]);

  function adicionarTarefa() {
    if (tarefa.trim() === '') return;

    // Criamos um objeto com id único baseado no timestamp atual
    setLista([...lista, { id: Date.now().toString(), texto: tarefa }]);
    setTarefa(''); // Limpamos o campo de texto
  }
}
```

```

return (
  <View style={styles.container}>
    <Text style={styles.titulo}>Minhas Tarefas</Text>

    <View style={styles.inputContainer}>
      <TextInput
        style={styles.input}
        placeholder="Digite uma tarefa..."
        onChangeText={({valor}) => setTarefa(valor)} // Atualiza o estado em tempo
real
        value={tarefa}
      />
      <Button title="ADICIONAR" onPress={adicionarTarefa} />
    </View>

    <FlatList
      data={lista} // Array de dados estruturados
      keyExtractor={({item}) => item.id} // Chave única identificadora
      renderItem={({ item }) => (
        <View style={styles.itemLista}>
          <Text>{item.texto}</Text>
        </View>
      )}
      ListEmptyComponent={<Text style={styles.empty}>Nenhuma tarefa cadastrada.
</Text>} // Feedback visual se vazio
    />
  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    paddingTop: 50, // Evita que o conteúdo fique sob o "notch" do celular
    backgroundColor: '#fff',
  },
  titulo: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
  inputContainer: {
    flexDirection: 'row',
    marginBottom: 20,
    gap: 10,
  },
  input: {
    flex: 1,
    borderWidth: 1,
    borderColor: '#ccc',
    padding: 10,
    borderRadius: 5,
  },
});

```

```

    },
    itemLista: {
      padding: 15,
      backgroundColor: '#e0e0e0',
      marginBottom: 10,
      borderRadius: 5,
    },
    empty: {
      textAlign: 'center',
      color: '#999',
      marginTop: 20,
    }
  }
});

```

Módulo 7 [Material 07]: Navegação entre Telas e Persistência Local (CineApp)

Avançando na arquitetura, dividimos nosso aplicativo em múltiplas janelas. Para gerenciar esse histórico, recorreremos ao **React Navigation**, utilizando especificamente o modelo de pilha (**Stack Navigator**).

1. A Metáfora da Pilha (Stack)

Pensem no Stack Navigator como um baralho de cartas. A tela inicial (**Home**) fica na base. Quando o usuário seleciona um item, colocamos uma nova carta (**Details**) por cima da pilha. O usuário passa a ver apenas o topo. Ao clicar no botão voltar (ou acionar um retorno programático por código), nós removemos (*pop*) a carta do topo, expondo novamente a tela anterior.

2. Persistência de Dados com AsyncStorage

Diferente do ambiente web onde usamos o **localStorage**, no ambiente mobile utilizamos o **@react-native-async-storage/async-storage** para salvar informações diretamente na memória do smartphone de forma permanente. Atendem-se às duas regras fundamentais do **AsyncStorage**:

1. **Ele trabalha estritamente no formato Chave-Valor e só aceita Strings.** Por conta disso, antes de salvar um objeto ou array complexo, convertemos para texto usando **JSON.stringify()**. Quando formos ler esse dado de volta, decodificamos a string para objeto original com **JSON.parse()**.
2. **Todas as suas operações são assíncronas (Promises).** Isso garante que a leitura e a escrita em disco rodem em segundo plano, sem travar ou congelar a interface gráfica do usuário enquanto os dados são processados.

Abaixo está a estruturação completa do nosso roteador centralizador (**App.js**), integrando a navegação nativa e passando parâmetros dinâmicos de uma tela para outra.

```

// App.js (CineApp)
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

```

```

import HomeScreen from './src/screen/HomeScreen';
import DetailsScreen from './src/screen/DetailsScreen';
import FavoritesScreen from './src/screen/FavoritesScreen';

const Stack = createNativeStackNavigator(); // Inicializa o gerenciador da pilha

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{ title: 'Cine App' }}
        />
        <Stack.Screen
          name="Details"
          component={DetailsScreen}
          options={{ title: 'Detalhes do Filme' }}
        />
        <Stack.Screen
          name="Favorites"
          component={FavoritesScreen}
          options={{ title: 'Meus Favoritos' }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

```

E aqui está o exemplo de como capturamos o parâmetro enviado pela rota dentro da tela de detalhes ([DetailsScreen.js](#)), realizando a checagem e a persistência na sequência:

```

// src/screen/DetailsScreen.js
import React from 'react';
import { View, Text, Button, Alert, StyleSheet } from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';

export default function DetailsScreen({ route, navigation }) {
  // Acessamos os dados passados através do objeto route.params
  const { filme } = route.params;

  const salvarFavorito = async () => {
    try {
      const favoritosAtuais = await AsyncStorage.getItem('@cinefatec_favoritos');
      let listaSalva = favoritosAtuais ? JSON.parse(favoritosAtuais) : [];

      // Evita duplicidade de IDs salvos na memória
      const jaExiste = listaSalva.find(f => f.id === filme.id);
      if (jaExiste) {
        Alert.alert("Aviso", "Este filme já está nos favoritos!");
      }
    }
  };
}

```

```

        return;
    }

    listaSalva.push(filme);
    // Transformamos a lista atualizada em String para que o armazenamento aceite
    await AsyncStorage.setItem('@cinefatec_favoritos',
    JSON.stringify(listaSalva));

    Alert.alert("Sucesso", "Filme salvo nos favoritos!");
    navigation.goBack(); // Força o retorno programático para a tela anterior
  } catch (e) {
    Alert.alert("Erro", "Não foi possível salvar o filme.");
  }
};

return (
  <View style={styles.container}>
    <Text style={styles.tituloGrande}>{filme.titulo}</Text>
    <Text style={styles.detalhe}>Ano: {filme.ano}</Text>
    <Text style={styles.sinopse}>{filme.sinopse}</Text>

    <View style={{ marginTop: 20 }}>
      <Button title="SALVAR NOS FAVORITOS" onPress={salvarFavorito} />
    </View>
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#fff' },
  tituloGrande: { fontSize: 28, fontWeight: 'bold', marginBottom: 10 },
  detalhe: { fontSize: 16, color: 'blue', marginBottom: 20 },
  sinopse: { fontSize: 16, lineHeight: 24, fontStyle: 'italic' }
});

```

Módulo 8 [Material 08]: Conectando com Banco de Dados na Nuvem — Backend as a Service (Supabase)

Evoluindo do armazenamento local isolado do **AsyncStorage**, integramos nosso projeto a um ecossistema real de produção utilizando o **Supabase**. Ele se consolida como uma excelente alternativa de código aberto ao Firebase, com o grande diferencial de ser inteiramente construído em cima do **PostgreSQL**, garantindo que trabalhemos com bancos de dados relacionais e robustos na nuvem.

1. O Ciclo do CRUD Completo

Com a biblioteca cliente do Supabase instalada, implementamos as 4 operações fundamentais de persistência em nuvem dentro do nosso app gerenciador de livros (**BookApp**):

- **CREATE (.insert):** Cadastramos registros enviando dados estruturados diretamente para as tabelas na nuvem.

- **READ (.select):** Consultamos os registros remotos, podendo aplicar ordenações explícitas no retorno (como ordenar de forma decrescente por ID).
- **UPDATE (.update):** Modificamos registros existentes aplicando filtros restritivos precisos como o `.eq('id', valor)` para atualizar estritamente a linha correta.
- **DELETE (.delete):** Removemos os dados de forma definitiva com base na identificação primária da linha.

2. O Desafio do Row Level Security (RLS) e useEffect

Por padrão de segurança do PostgreSQL na nuvem, as tabelas criadas vêm travadas. Para permitir consultas e inserções do nosso app, precisamos configurar as regras de **RLS (Row Level Security)** habilitando privilégios públicos ou específicos de leitura e escrita.

Outro detalhe crucial de arquitetura: quando abrimos a tela inicial, o hook tradicional `useEffect` roda o carregamento de dados apenas uma vez. Porém, se navegamos para a tela de formulário, inserimos ou editamos um livro e depois voltamos à tela principal, o `useEffect` não é disparado de novo por padrão. Para contornar isso e fazer com que a listagem atualize as informações automaticamente sempre que a janela ganhar o foco visual do usuário, recorreremos ao hook `useFocusEffect` combinado com o `useCallback` do React Native.

Vejam abaixo a implementação da nossa listagem principal integrada com as funções assíncronas do Supabase (`src/screen/HomeScreen.js`):

```
// src/screen/HomeScreen.js (BookApp)
import React, { useState, useCallback } from 'react';
import { View, Text, FlatList, StyleSheet, TouchableOpacity, Alert,
ActivityIndicator } from 'react-native';
import { useFocusEffect } from '@react-navigation/native';
import { supabase } from '../services/supabase';

export default function HomeScreen({ navigation }) {
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(true);

  // OPERAÇÃO READ: Busca as informações em tempo real na nuvem
  const fetchBooks = async () => {
    setLoading(true);
    const { data, error } = await supabase
      .from('books')
      .select('*')
      .order('id', { ascending: false }); // Garante os registros mais recentes no
    topo

    if (error) {
      Alert.alert("Erro", error.message);
    } else {
      setBooks(data);
    }
    setLoading(false);
  };
}
```

```

// Garante o re-fetch e atualização dos dados toda vez que a tela ganhar foco
useFocusEffect(
  useCallback(() => {
    fetchBooks();
  }, [])
);

// OPERAÇÃO DELETE: Remove o item com base no ID recebido
const handleDelete = (id) => {
  Alert.alert("Excluir", "Tem certeza que deseja apagar este livro?", [
    { text: "Cancelar", style: "cancel" },
    {
      text: "Apagar",
      style: "destructive",
      onPress: async () => {
        const { error } = await supabase.from('books').delete().eq('id', id);
        if (error) {
          Alert.alert("Erro", error.message);
        } else {
          fetchBooks(); // Recarrega a listagem após a exclusão
        }
      }
    }
  ], []);
};

return (
  <View style={styles.container}>
    {loading ? (
      <ActivityIndicator size="large" color="#007AFF" style={{ marginTop: 20 }} />
    ) : (
      <FlatList
        data={books}
        keyExtractor={(item) => item.id.toString()}
        renderItem={({ item }) => (
          <View style={styles.card}>
            <View style={styles.textContainer}>
              <Text style={styles.title}>{item.title}</Text>
              <Text style={styles.author}>{item.author}</Text>
            </View>
            <View style={styles.actions}>
              <TouchableOpacity onPress={() => navigation.navigate('BookForm', {
book: item })}>
                <Text style={{ color: 'green', fontWeight: 'bold' }}>Editar</Text>
              </TouchableOpacity>
              <TouchableOpacity onPress={() => handleDelete(item.id)}>
                <Text style={{ color: 'red', fontWeight: 'bold' }}>Excluir</Text>
              </TouchableOpacity>
            </View>
          </View>
        )}
      </FlatList>
      ListEmptyComponent={<Text style={styles.emptyText}>Nenhum livro
cadastrado.</Text>}
    )}
  </View>
);

```

```

    })

    { /* Botão Flutuante (FAB) para Adicionar */ }
    <TouchableOpacity style={styles.fab} onPress={() =>
navigation.navigate('BookForm')}>
      <Text style={styles.fabText}></Text>
    </TouchableOpacity>
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5', padding: 10 },
  card: {
    flexDirection: 'row',
    backgroundColor: '#fff',
    padding: 15,
    borderRadius: 8,
    marginBottom: 10,
    alignItems: 'center',
    elevation: 2
  },
  textContainer: { flex: 1 },
  title: { fontSize: 18, fontWeight: 'bold', color: '#333' },
  author: { fontSize: 14, color: '#666' },
  actions: { flexDirection: 'row', gap: 15 },
  emptyText: { textAlign: 'center', marginTop: 50, color: '#999' },
  fab: {
    position: 'absolute',
    bottom: 20,
    right: 20,
    backgroundColor: '#007AFF',
    width: 56,
    height: 56,
    borderRadius: 28,
    justifyContent: 'center',
    alignItems: 'center',
    elevation: 5
  },
  fabText: { color: '#fff', fontSize: 28, fontWeight: 'bold' }
});

```

Módulo 9 [Material 09]: Comunicação e Arquitetura em Tempo Real (WebSockets)

Para fechar com chave de ouro, estudamos como criar experiências dinâmicas contornando as limitações dos alertas tradicionais via **Tecnologia Realtime baseada em WebSockets**.

1. Polling vs. Listening

- **Polling (Mecanismo Tradicional):** O aplicativo age de forma "chata", fazendo requisições repetitivas ao servidor de tempos em tempos (ex: a cada 5 segundos) perguntando: "*Tem dados novos? E agora, tem dados novos?*". Isso gera um consumo massivo de processamento, dados móveis e bateria.
- **Listening / Streaming (WebSockets):** Estabelecemos uma conexão única persistente aberta bidirecional. O aplicativo fica em repouso escutando os eventos. Quando um evento acontece no banco de dados, o próprio servidor toma a iniciativa de "empurrar" o payload com a alteração instantaneamente para todos os dispositivos conectados na rede.

No nosso projeto *EngCompAlert*, quando o administrador realiza um `.insert` inserindo uma nova linha na tabela `notifications` na nuvem, a engine de Realtime do Supabase captura o evento e faz um broadcast imediato via socket para os celulares dos alunos. Ao capturar esse evento em background, o aplicativo móvel utiliza a biblioteca `expo-notifications` para disparar um alerta sonoro e visual local imediato, mesmo que a tela estivesse travada em outra operação.

Abaixo está o trecho do ciclo de vida que injetamos dentro do `useEffect` principal do nosso `App.js` para ligar o canal de escuta remota da aplicação:

```
// Trecho de Código de Inicialização Realtime em App.js
useEffect(() => {
  // A) Solicita as permissões do sistema operacional para emitir som e pop-ups
  visuais
  registerForLocalNotificationsAsync().then(granted => {
    setPermissionGranted(granted);
  });

  // B) Conecta à engrenagem de WebSockets e liga o "Radar" do Supabase Realtime
  const subscription = supabase
    .channel('public:notifications') // Define o escopo do canal público
    .on(
      'postgres_changes',
      { event: 'INSERT', schema: 'public', table: 'notifications' }, // Escuta
      exclusivamente novos INSERTS
      (payload) => {
        console.log("Mágica Realtime! Dado detectado na nuvem:", payload.new);

        // C) Dispara localmente o alerta sonoro e visual com as informações vindas
        do banco
        triggerLocalNotification(payload.new.title, payload.new.body);
      }
    )
    .subscribe(); // Ativa e inicia a escuta ativa da conexão

  // Método de limpeza (cleanup): Remove e encerra o canal socket aberto ao fechar o
  app
  return () => {
    supabase.removeChannel(subscription);
  };
}, []);
```

Dicas do Professor para a Avaliação:

1. **Atenção aos nomes das propriedades e métodos assíncronos:** Não se esqueçam de utilizar as palavras-chave `async` e `await` sempre que estiverem lidando com chamadas externas (`AsyncStorage` e funções do `supabase`).
2. **Configuração de tabelas:** Lembrem-se que no Supabase, se criarmos uma tabela e esquecermos de marcar explicitamente a opção "**Enable Realtime**", o banco de dados funcionará normalmente no fluxo CRUD, mas as atualizações em tempo real via WebSockets nunca chegarão aos aplicativos conectados.
3. **Diferencie componentes conceituais:** Saibam explicar claramente o motivo de usarmos componentes estruturados performáticos como o `<FlatList>` e o hook de ciclo de vida avançado `useFocusEffect` no lugar de suas contrapartidas tradicionais da web.

Estudem bem esses blocos estruturais e revisem as analogias práticas que estabelecemos ao longo do semestre. Tenho certeza de que todos vocês se sairão bem.

Nos vemos na avaliação! Um abraço e ótimos códigos!